Project Phase-I Report

On

Cloud Cost Prediction and Optimization

Submitted by

Sathe Onkar Laxman
Pakdhane Sujal Chandrakant
Pawar Sourabh Vinayak

Under the guidance of

Dr. V. H. Kondekar



Department of Electronics and Telecommunication Engineering

S.A.P.D. J. Pathashala's
Walchand Institute of Technology, Solapur
(An Autonomous Institute)

2026–27

CERTIFICATE

This is to certify that the Project Phase-I Report entitled

Cloud Cost Prediction and Optimization

Submitted by

Name	Roll. No.	Exam Seat No.
Sathe Onkar Laxman	13	2304111113
Pakdhane Sujal Chandrakant	14	2304111062
Pawar Sourabh Vinayak	15	2304111050

has been approved as partial fulfillment for the award of the degree in B.Tech Electronics and Telecommunication Engineering at Walchand Institute of Technology, Solapur, affiliated to Punyashlok Ahilyadevi Holkar Solapur University, Solapur, in the academic year 2026–27.

Project Guide	**Head, E&TC Engineering**
Dr. V. H. Kondekar	Dr. Ms. A. V. Thalange

Dr. S. R. Gengaje
I/C Principal

Department of Electronics and Telecommunication Engineering
S.A.P.D. J. Pathashala's
Walchand Institute of Technology, Solapur
(An Autonomous Institute)
2026–27

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, Dr. V. H. Kondhekar, for valuable guidance, encouragement, and constructive suggestions throughout the planning and development of the project titled “Cloud Cost Prediction and Optimization.” His guidance helped us understand the problem, organize the implementation into phases, and maintain attention to safe and responsible resource-management practices.

We are thankful to the Head of the Department of Electronics and Telecommunication Engineering and the faculty members of Walchand Institute of Technology, Solapur, for providing academic support and the opportunity to undertake this project as part of our engineering studies.

We also acknowledge the maintainers of the software libraries, cloud documentation, research publications, and development tools consulted during the project. These resources supported our understanding of cost-data collection, time-series analysis, database design, API development, and controlled remediation.

INDEX

Chapter	Name of Chapter	Page No.
	Abstract	6
Chapter 1	Introduction	7
1.1	Introduction	7
1.2	Purpose of the Project	8
Chapter 2	Literature Survey	9
2.1	Literature Survey: Papers and Products	9
2.2	Research Gap	11
2.3	Problem Statement	11
2.4	Objectives	11
2.5	Scope of the Project	12
2.6	Sustainable Development Goals Addressed	12
Chapter 3	System Description	13
3.1	System Description and Block Diagram	13
3.2	Methodology	14
3.3	Complexity and Feasibility of Design	16
Chapter 4	System Implementation	17
4.1	System Implementation	17
4.2	Results (Status to Date)	18
Chapter 5	Resource Management and Budget Estimate	21
5.1	Hardware Components and Specification	19
5.2	Software Tools and Technology Stack	20
5.3	Resource Management	22
5.4	Budget Estimate	23

Chapter 6	Advantages, Disadvantages and Applications	24
Chapter 7	Conclusion and Future Scope	26
7.1	Conclusion	26
7.2	Future Scope	27
Chapter 8	Time Line Chart of Project	28
	Bibliography	29
	Appendix	31

List Of Tables

Table. No.	Title of Table	Page No.
2.6	Sustainable Development Goals Addressed	12
3.2	Database Design	14
4.1	System Implementation	17
4.2	Result (Status of date)	20
5.1	Hardware Components and Specification	21
5.2	Software Tools and Technology Stack	22
5.4	Budget Estimate	23
8.1	Project Timeline	28

List Of Figures

Figure No.	Title of Figure	Page No.
3.1	System Description and Block Diagram	14
4.1	Main Dashboard	19
4.2	Result Dashboard	20

ABSTRACT

Cloud Cost Prediction and Optimization is a software project intended to help users understand cloud expenditure, examine daily cost trends, identify unusual spending, and review resources that may be idle or stopped. Cloud billing records can be difficult to interpret when information is distributed across different services and presented in provider-specific formats. The proposed application addresses this issue by collecting cost records through provider adapters, converting them into a normalized internal structure, and storing them in a relational database for analysis.

The application uses Python and FastAPI to expose REST endpoints, PostgreSQL to persist accounts and cost records, SQLAlchemy for database access, and Alembic for schema migrations. Cost analysis prepares daily time series, generates future cost estimates using Prophet when sufficient history is available, and uses a linear-trend fallback when Prophet cannot be applied. A rolling upper-bound rule flags unusually high daily costs. A browser dashboard presents cost records, forecasts, anomalies, idle resources, and action logs.

The project also includes a controlled remediation workflow. Stop or delete operations are not performed automatically: the request must contain explicit approval, the resource must meet eligibility checks, and each action is recorded with its status and provider response. A sample-data mode supports demonstrations and development without cloud credentials or changes to real resources. The current implementation is organized into phases covering application foundation, optional infrastructure configuration, ingestion, analytics, and dashboard/remediation. This report describes the motivation, related work, system design, methodology, implementation status, resources, limitations, and future work.

Keywords: cloud cost analysis, cost forecasting, anomaly detection, FastAPI, PostgreSQL, Prophet, resource management, audit logging.

CHAPTER 1: INTRODUCTION

1.1 Introduction

Cloud computing provides on-demand access to computing, storage, networking, and managed services. Its flexible usage model helps teams provision resources quickly, but it also makes expenditure dependent on resource configuration, workload behaviour, service selection, and the duration for which resources remain active. As systems evolve, the amount of billing information can become difficult to review manually.

Cloud Cost Prediction and Optimization is designed as a decision-support application for cost visibility and resource review. It gathers daily cost records through provider-specific adapters and represents them using common fields such as date, service, region, amount, currency, and account. This normalized representation supports consistent storage and analysis without requiring every downstream service to understand the original provider response format.

The application is organized around five functional areas: ingestion, persistence, analytics, presentation, and controlled remediation. Ingestion collects or simulates billing records and resource candidates. PostgreSQL stores the normalized records. The analytics service prepares time series, predicts future values, and detects unusually high observations. The browser dashboard displays results to a user. Finally, a controlled workflow allows eligible resources to be stopped or deleted only after explicit approval and records the outcome for auditing.

The project uses Python 3.13, FastAPI, Pydantic, SQLAlchemy, PostgreSQL, Alembic, Prophet, pandas, provider SDKs, HTML, CSS, JavaScript, Docker Compose, Terraform, and pytest. These tools provide a modular structure for API development, data storage, analysis, local deployment, optional infrastructure setup, and automated testing.

1.2 Purpose of the Project

The main purpose is to develop a practical application that converts cloud billing and resource information into understandable summaries and forecasts. It is intended to help users review historical costs, understand expected spending patterns, identify unusual daily charges, and examine resources that may be generating charges without being actively used.

The project has technical relevance because it combines API integration, database design, data preprocessing, time-series forecasting, anomaly detection, dashboard development, and secure action validation in one software workflow. Its social and organizational relevance lies in supporting

informed technology budgeting and responsible use of computing resources. Its environmental relevance is indirect: better visibility into resource usage can support decisions about unnecessary infrastructure, although the project does not itself measure environmental impact or carbon emissions.

The project was selected because cloud billing analysis is a real software-engineering problem that benefits from automation and structured data processing. It also provides an opportunity to apply concepts from software architecture, databases, statistical analysis, web services, and software testing in a single engineering project.

CHAPTER 2: LITERATURE SURVEY

2.1 Literature Survey: Papers and Products

The review covers foundational forecasting research, anomaly-detection studies, cloud cost-management products, and the technical documentation of the libraries and services used by the project. These sources inform the design choices; they are not presented as direct proof of the performance of this implementation.

1. Hyndman and Athanasopoulos, Forecasting: Principles and Practice (2021). Explains time-series decomposition, forecasting evaluation, and prediction intervals. It provides background for treating daily billing values as an ordered series and for communicating forecast uncertainty.

2. Taylor and Letham, "Forecasting at Scale," The American Statistician (2018). Introduces Prophet, a forecasting procedure that models trend, seasonality, and special events. The project uses Prophet as the preferred forecasting option when sufficient observations and dependencies are available.

3. Box, GitHub Actions, Reinsel and Ljung, Time Series Analysis: Forecasting and Control (2015). Presents classical time-series methods and model diagnostics. It is relevant background for understanding that forecast quality depends on data behaviour and validation.

4. Chandola, Banerjee and Kumar, "Anomaly Detection: A Survey," ACM Computing Surveys (2009). Surveys anomaly-detection approaches and application domains. It supports the distinction between normal observations and observations that differ from an established baseline.

5. Hodge and Austin, "A Survey of Outlier Detection Methodologies," Artificial Intelligence Review (2004). Reviews statistical and machine-learning approaches to outlier detection. It provides a basis for choosing a transparent threshold rule for an initial project implementation.

6. Ahmad, Lavin, Purdy and Agha, "Unsupervised Real-Time Anomaly Detection for Streaming Data," Neurocomputing (2017). Discusses streaming anomaly detection and the importance of learning patterns from historical data. It is relevant to future extensions for continuously arriving billing observations.

7. Nawrocki and Sus, "Anomaly detection in the context of long-term cloud resource usage planning," Knowledge and Information Systems (2022). Studies anomaly detection for cloud-resource planning. It is directly relevant to the project goal of identifying cost or usage behaviour that warrants investigation.

8. Smendowski and Nawrocki, "Optimizing multi-time series forecasting for enhanced cloud resource utilization based on machine learning," Knowledge-Based Systems (2024). Explores forecasting and anomaly detection for cloud resource planning. It highlights the value of considering series context and model uncertainty when using predictions to support resource decisions.

9. "Intrusion detection in cloud computing based on time series anomalies utilizing machine learning," Journal of Cloud Computing (2023). Uses time-series methods and Prophet in a cloud anomaly-detection context. Although its target is security events rather than billing, it illustrates how time-series analysis can help identify unusual cloud-system behaviour.

10. Meta Prophet documentation. Documents model configuration, seasonality, forecast outputs, and uncertainty intervals. It is the primary implementation reference for the project's preferred forecasting model.

11. Amazon Web Services Cost Explorer API documentation. Describes programmatic access to aggregated cost and usage data. The project's AWS adapter uses Cost Explorer for daily service and region cost records.

12. Microsoft Azure Cost Management documentation. Describes querying and exporting cost information at supported scopes. It informs the Azure cost-collection adapter and the need for correct permissions and billing configuration.

13. Google Cloud Billing export to BigQuery documentation. Explains how billing records can be exported into BigQuery for analysis. The project reads cost information from a configured billing-export table.

14. AWS Trusted Advisor / Cost Optimization guidance. Provides recommendations for examining cloud-resource usage and cost-related configuration. It represents a provider-native approach that users may compare with the project's consolidated review workflow.

15. Azure Advisor documentation. Describes recommendations for reliability, performance, security, and cost. It illustrates the value of presenting resource recommendations alongside billing analysis.

16. Google Cloud Recommender documentation. Describes recommendation services that help identify configuration and resource-management opportunities. It is relevant as a possible source of external recommendations in future work.

17. FinOps Foundation, FinOps Framework. Presents collaborative practices for financial accountability in cloud use. It supports the project's focus on making cost information visible and actionable for technical decision-makers.

18. FastAPI documentation. Explains request validation, routing, automatic API schemas, and interactive API documentation. FastAPI is used for the versioned REST interface.

19. SQLAlchemy 2.0 documentation. Documents ORM models, database sessions, and persistence patterns. It informs the relational model and service-to-database interaction.

20. PostgreSQL documentation. Describes relational storage, constraints, transactions, and SQL operations. PostgreSQL stores accounts, daily costs, resources, forecasts, anomalies, and remediation logs.

21. Alembic documentation. Explains repeatable database schema migrations. The project includes migration support so schema changes can be managed in a controlled way.

22. pandas documentation. Provides tools for grouping, sorting, date indexing, and preparing data for time-series analysis. The analytics pipeline uses these types of data-preparation operations.

23. Docker Compose documentation. Describes the configuration and lifecycle of multi-container local applications. It is used to run the API and PostgreSQL together during development.

24. HashiCorp Terraform documentation. Documents infrastructure-as-code workflows and validation. The project includes optional sandbox configuration with conservative defaults; infrastructure creation is not required for sample-mode demonstrations.

25. pytest documentation. Describes automated test discovery, fixtures, and assertions. It supports the project's API and contract-test strategy.

2.2 Research Gap

Provider-native billing tools and general forecasting libraries provide useful individual capabilities, but a project-level workflow still needs to connect data collection, normalization, persistence, forecasting, anomaly review, resource eligibility checks, and action auditing. The reviewed sources do not by themselves define the exact data model, API contract, approval policy, or dashboard required for this project.

The project addresses this integration gap at an educational prototype level by defining a common cost-record structure, keeping provider-specific logic inside adapters, storing analytical outputs, and requiring explicit approval before a resource-changing operation. The system does not claim that its

thresholds or forecasts are universally optimal; they are transparent starting points that require evaluation with representative billing histories.

2.3 Problem Statement

To design and implement a software system that collects or simulates daily cloud cost records, stores them in a normalized database, forecasts near-term spending, detects unusually high daily costs, displays findings in a browser dashboard, and supports safe, approval-controlled actions for eligible idle or stopped resources with a persistent audit trail.

2.4 Objectives

- Design a modular application architecture with a versioned REST API.
- Define normalized database entities for accounts, cost records, resources, forecasts, anomalies, and remediation logs.
- Collect or simulate daily cost and resource information through adapter interfaces.
- Prepare continuous daily time series for analytical processing.
- Generate future cost estimates using Prophet when appropriate and a linear-trend fallback otherwise.
- Identify unusually high daily costs using a rolling baseline and a transparent upper threshold.
- Present costs, forecasts, anomalies, resources, and action history through a browser dashboard.
- Require explicit approval and eligibility validation before stop or delete operations.
- Support local development through sample data, Docker Compose, migrations, and automated tests.

2.5 Scope of the Project

The current scope includes a FastAPI application, PostgreSQL persistence, normalized provider adapters, sample data, cost forecasting, threshold-based anomaly detection, a browser dashboard, and approval-controlled remediation. The repository also contains optional Terraform configuration for sandbox infrastructure. Sample mode is intended for development and demonstration without cloud credentials.

The project has important limitations. Forecast quality depends on the amount, consistency, and representativeness of historical data. The linear fallback is intentionally simple and may not capture complex seasonal behaviour. The anomaly rule is a transparent heuristic rather than a fully trained classifier. Real provider integrations require valid credentials, permissions, billing configuration, and

network access. Remediation actions can affect infrastructure and must be used only after careful review. The application is an educational prototype and is not a substitute for provider-native controls, financial review, or organization-specific approval policies.

2.6 Sustainable Development Goals Addressed

SDG	Connection to the project
SDG 9: Industry, Innovation and Infrastructure	Demonstrates software methods for monitoring and managing digital infrastructure and its usage data.
SDG 12: Responsible Consumption and Production	Supports informed review of resource usage and avoidable infrastructure activity; environmental impact is not directly measured.
SDG 13: Climate Action (indirect relevance)	Resource visibility may support operational decisions that consider efficiency, but the application does not calculate carbon emissions or claim measured climate benefits.

CHAPTER 3: SYSTEM DESCRIPTION

3.1 System Description and Block Diagram

The system accepts requests from a user, browser dashboard, or API client. FastAPI validates requests and forwards them to application services. Provider adapters collect billing records and resource candidates, while the sample adapter supplies predictable test data. Records are normalized and persisted in PostgreSQL through SQLAlchemy. The analytics service reads daily cost history, creates forecasts and anomaly records, and returns results to the dashboard. Remediation requests pass through approval and eligibility checks before a provider action is attempted.

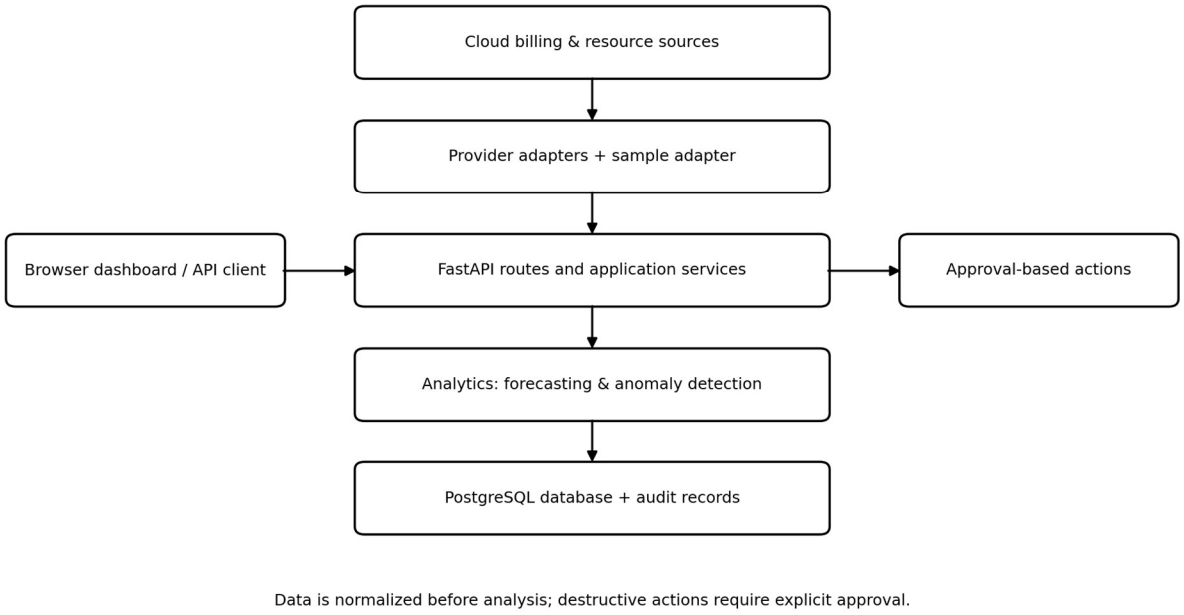


Figure 3.1: System Description and Block Diagram

Main modules

- API layer: versioned routes, input validation, and documented request/response schemas.
- Adapter layer: isolates provider SDK calls and converts provider-specific data into common internal records.
- Ingestion service: collects cost data, manages the requested date range, and inserts or updates resource records.
- Analytics service: groups cost records, fills missing dates, generates forecasts, and stores anomalies.

- Persistence layer: SQLAlchemy models and PostgreSQL tables provide durable storage.
- Dashboard: HTML, CSS, and JavaScript interface for synchronization, analysis, visualizations, and remediation review.
- Remediation service: validates the resource, requested action, and explicit approval; records success or failure in an audit log.

3.2 Methodology

3.2.1 Data ingestion and normalization

An ingestion request identifies the selected provider adapters, date range, whether resource discovery is required, and whether sample data should be used. If dates are omitted, the API applies its configured default window. For each provider, the service creates or reuses an account record, collects cost observations, replaces records only within the selected account/date range to avoid stale duplicates, and inserts or updates resource candidates using their provider resource identifiers. Provider-level errors are reported separately so that a failure in one connection does not hide results from another.

3.2.2 Database design

Entity / Table	Purpose
cloud_accounts	Stores normalized account or subscription identity for a connected provider.
cost_records	Stores daily cost by account, service, region, amount, currency, date, and dimensions.
cloud_resources	Stores resource identifier, type, region, status, tags, metadata, idle reason, and estimated monthly cost when available.
cost_forecasts	Stores forecast date, predicted cost, lower and upper bounds, and the model used.
cost_anomalies	Stores actual cost, expected cost, threshold, severity, explanation, and calculation metadata.
remediation_logs	Stores requested action, requester, status, provider response, error details, and timestamps.

3.2.3 Forecasting algorithm

Cost records are grouped by account, service, region, and currency. Observations on the same day are summed, the series is sorted by date, and missing dates between the first and last observation are filled with zero values. Prophet is preferred when the required dependency is available, the model is enabled, and the series contains sufficient history. Prophet models trend and weekly seasonality and returns predicted values with uncertainty bounds. If it is unavailable, cannot fit the data, or the history is too short, the application uses a linear-trend fallback with statistically derived bounds.

The forecast output includes the forecast date, predicted cost, lower and upper bounds, series identifiers, and model name. These values are intended as estimates rather than guaranteed future bills. A responsible deployment should compare forecasts with held-out historical periods and report suitable error measures before relying on them for budget decisions.

3.2.4 Anomaly detection

For each eligible daily observation, the detector calculates a baseline from preceding daily costs. The expected value is based on the preceding mean. The upper threshold is the maximum of the expected value plus a statistical margin, a fixed relative margin above the expected value, and a small minimum threshold. If actual cost exceeds the threshold, an anomaly record is stored. Severity is assigned according to how far the actual value exceeds the upper bound. Because the rule is explicit, a user can inspect the threshold and explanation; however, it can still produce false positives or miss patterns that do not exceed the threshold.

3.2.5 Controlled remediation

The user reviews the resource list and selects an action supported for that resource type. The request must include explicit approval. The service rejects unsupported actions, missing resources, and resources that do not satisfy the required idle or stopped-state checks. The provider response and outcome are persisted in remediation logs. Sample mode simulates the operation and does not change real infrastructure. Deletion is potentially irreversible, so the workflow should be used only with verified resource identity, appropriate access control, and organizational approval.

3.3 Complexity and Feasibility of Design

For N daily cost records, grouping and sorting the records is approximately O(N log N) when sorting is required. A linear pass over an already ordered series for rolling statistics or threshold checks is O(N). A forecast operation depends on the selected model, series length, model settings, and hardware; therefore, no fixed runtime is claimed in this report. Database query cost depends on indexes, filters, and record volume.

The design is technically feasible for a prototype because its components use established libraries and standard HTTP/database interfaces. Sample mode makes local demonstration possible without active provider credentials. Docker Compose simplifies starting the API and database. Terraform is optional and uses disabled-by-default provider/compute settings to reduce the chance of accidental resource creation. Operational feasibility still depends on environment configuration, permissions, data quality, and careful review of actions.

CHAPTER 4: SYSTEM IMPLEMENTATION

4.1 System Implementation

Phase	Implementation work	Status described by project files
Phase 1: Foundation	FastAPI application, health route, SQLAlchemy models, PostgreSQL setup, Alembic migration, adapter contract, Docker files, contract tests.	Implemented foundation; provider actions are placeholders at this stage.
Phase 2: Sandbox infrastructure	Terraform modules and shared naming/tagging conventions; providers and compute instances disabled by default.	Configuration validated; project notes state that no cloud plan/apply was run and no resources were created.
Phase 3: Ingestion	Provider adapters, sample adapter, normalized cost/resource records, ingestion service, API endpoints.	Implemented for configured provider integrations and sample mode; real data depends on credentials and billing setup.
Phase 4: Analytics	Daily time-series preparation, Prophet forecast path, linear-trend fallback, threshold-based anomaly detection, API routes.	Implemented according to the phase notes; output quality depends on input history and model conditions.
Phase 5: Dashboard and remediation	Browser dashboard, idle-resource listing, approval-controlled stop/delete requests, provider-response persistence, audit logs.	Workflow implemented; live actions require correctly configured credentials and careful approval.

Application endpoints

Method	Endpoint	Purpose
GET	/api/v1/health	Checks application health.
POST	/api/v1/ingestion/sync	Collects or simulates cost records and optionally resource candidates.
GET	/api/v1/ingestion/costs	Lists stored cost records.
POST	/api/v1/analysis/run	Runs forecast and anomaly analysis.
GET	/api/v1/analysis/forecasts	Lists stored forecast results.
GET	/api/v1/analysis/anomalies	Lists detected anomalies.
GET	/api/v1/remediation/resources	Lists resources eligible for review.
POST	/api/v1/remediation/resources/{resource_id}	Requests an approved stop or delete action.
GET	/api/v1/remediation/logs	Lists remediation audit records.

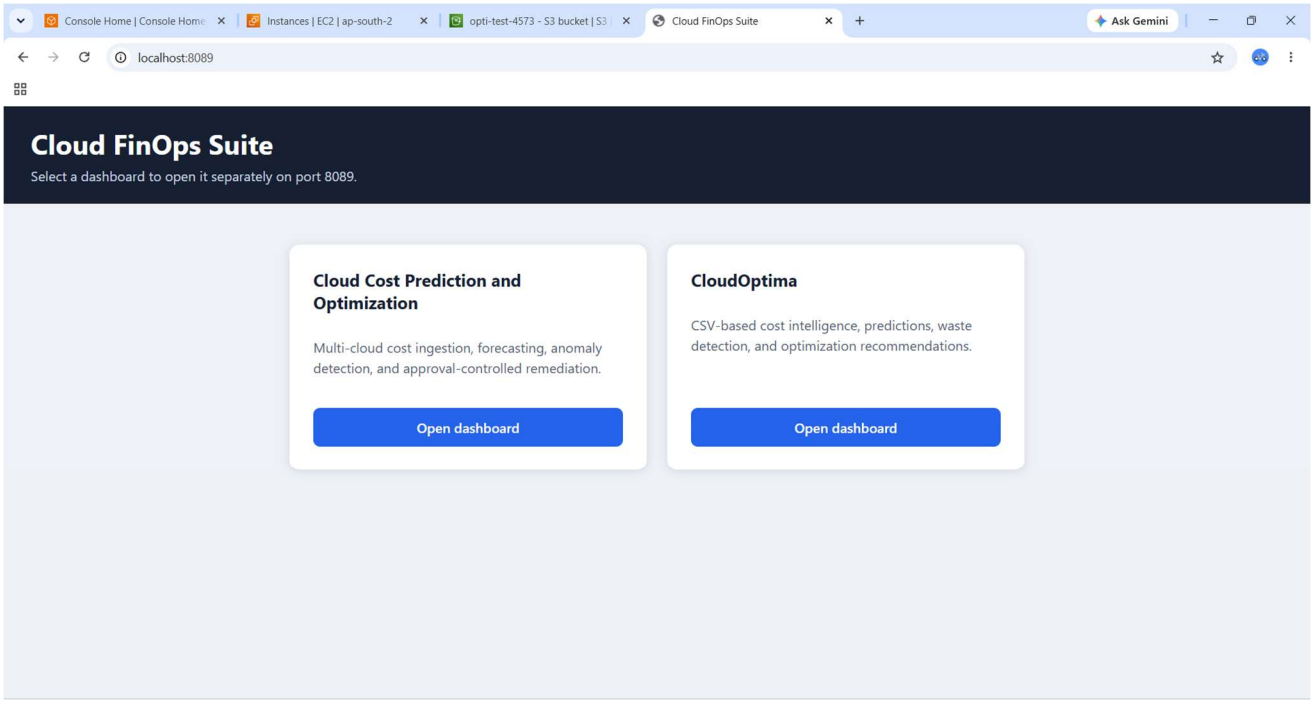


Fig. 4.1 Main Dashboard

Local execution approach

For local development, the project requires Python 3.13 and PostgreSQL 16, or Docker Desktop with Docker Compose. The environment file should be copied from the supplied example and configured for the local database. Alembic migrations are the repeatable schema-management method. The API can be started with Uvicorn, and the dashboard is served from the application root. Interactive API documentation is available through the FastAPI documentation route.

4.2 Results (Status to Date)

The phase documentation records implementation of the API foundation, normalized database schema, adapter contracts, optional Terraform sandbox modules, data-ingestion services, analytical services, dashboard, and controlled remediation workflow. The phase notes also describe validation of the Terraform configuration and the existence of automated tests. This report does not state a test pass count or forecast-accuracy score because such numeric results were not provided in the source materials.

Sample mode is available to demonstrate ingestion, analysis, dashboard rendering, and simulated remediation without cloud credentials. In this mode, resource actions are simulated and do not change real infrastructure. For live-provider results, the user must configure valid credentials and billing

permissions, run ingestion, and inspect returned provider-specific statuses. Forecast quality and anomaly usefulness should be assessed using representative historical data and a documented validation procedure.

Area	Evidence / expected observable output
Health API	Returns application status and environment information.
Ingestion	Returns provider-level outcomes and counts of collected cost/resource records.
Forecasting	Stores daily forecast values with bounds and the selected model identifier.
Anomaly detection	Stores observations exceeding the calculated upper threshold, including severity and explanation.
Remediation	Requires approval and records action status, provider response, or failure details.
Dashboard	Displays cost history, forecasts, anomalies, resource candidates, and remediation logs.

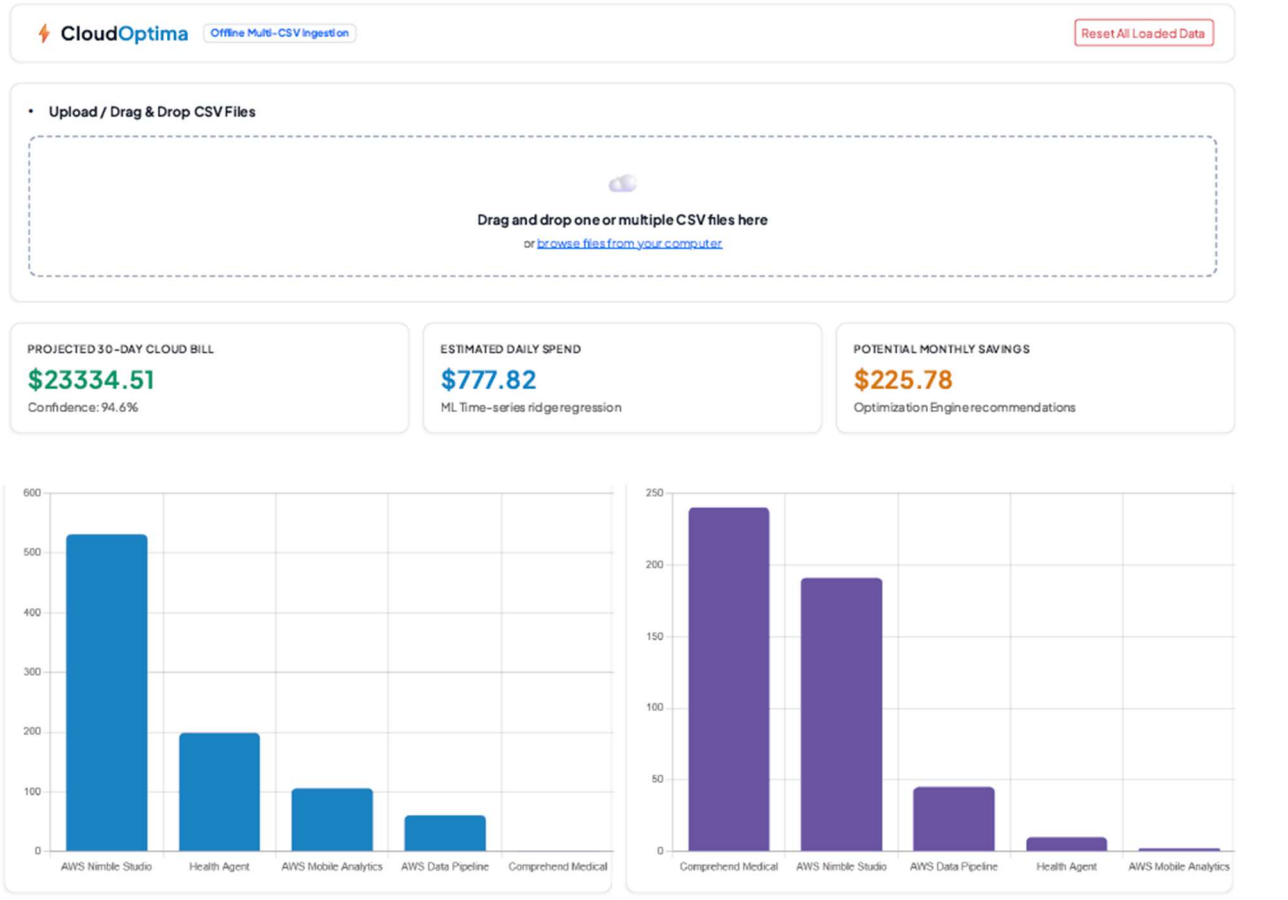


Fig 4.2 Result Dashboard

CHAPTER 5: RESOURCE MANAGEMENT AND BUDGET ESTIMATE

5.1 Hardware Components and Specification

The project is software-oriented and does not require a dedicated hardware circuit or specialized embedded board. A standard development computer with internet access is sufficient for local development. Actual performance depends on dataset size, database workload, and the selected forecasting model.

Component	Recommended / expected specification	Purpose
Development computer	Modern 64-bit laptop or desktop; multi-core CPU; 8 GB RAM or more recommended; sufficient disk space for Python packages and containers.	Development, running the API/database, and testing.
Network connection	Internet access when installing packages or connecting to live provider APIs.	Package installation and optional live data collection.
Cloud account access	Optional provider account with least-privilege permissions for cost queries and resource discovery.	Live ingestion and approved resource actions.
Display / browser	Current desktop browser.	Accessing the dashboard and API documentation.

5.2 Software Tools and Technology Stack

Tool / technology	Role in the project
Python 3.13	Primary application language.
FastAPI and Uvicorn	REST API framework and ASGI server.
Pydantic / pydantic-settings	Request/response validation and environment configuration.
SQLAlchemy 2	Object-relational mapping and database access.
PostgreSQL 16	Persistent relational database.
Alembic	Versioned database schema migrations.
Prophet	Preferred daily time-series forecasting model when sufficient history is available.
pandas, Python statistics, Decimal	Data preparation, fallback calculations, thresholds, and currency-safe handling.
boto3, Azure SDKs, Google Cloud SDKs	Provider-specific cost retrieval, resource discovery, and supported actions.
HTML, CSS, JavaScript	Browser dashboard user interface.
Docker and Docker Compose	Repeatable local API/database environment.
Terraform 1.6+	Optional infrastructure-as-code sandbox configuration.
pytest and httpx	Automated testing and HTTP endpoint tests.

5.3 Resource Management

The project manages resources in both the software-development environment and the connected cloud accounts. Software resources include source code, environment configuration, database migrations, dependencies, sample records, and test fixtures. These should be version-controlled where appropriate, while secrets and local environment files should be excluded from source control.

For live integrations, provider credentials should use the provider's recommended identity mechanism and least-privilege permissions. Access to billing information should be limited to the users and services that require it. Resource-changing operations must be reviewed before approval. The application stores action outcomes in an audit log to support traceability. Terraform sandbox settings default providers and optional compute instances to disabled, and infrastructure should not be created unless a deliberate plan has been reviewed.

5.4 Budget Estimate

The following is a planning estimate for a student project using existing personal or college equipment and local sample data. It is not a vendor quotation. Actual costs depend on whether paid cloud resources, additional storage, or commercial services are used.

Item	Estimated cost (INR)	Remarks
Existing laptop / desktop	₹0 incremental	Assumes an existing development computer is available.
Open-source software libraries	₹0 licence fee	Python, FastAPI, PostgreSQL, Prophet, SQLAlchemy, Docker, Terraform, and pytest are available as open-source tools; usage and hosting costs may still apply.
Local database and sample-mode testing	₹0 incremental	Assumes execution on an existing computer.
Internet / electricity	Existing household or institutional plan	Incremental amount depends on usage and local tariff.
Optional cloud sandbox usage	Variable; set a strict budget before use	Live API calls, storage, queries, and provisioned resources may incur charges. Keep compute disabled unless needed and approved.

The project can be developed and demonstrated in sample mode without provisioning paid cloud compute. Before enabling live integrations or applying infrastructure changes, the team should review the provider pricing calculator, configure spending alerts where available, and remove temporary resources after use.

CHAPTER 6: ADVANTAGES, DISADVANTAGES AND APPLICATIONS

Advantages

- Provides a structured view of cost records by date, service, region, account, and currency.
- Uses a common internal data model to separate provider-specific logic from analysis services.
- Supports forecasting and makes the selected model and forecast bounds available for review.
- Uses an explainable upper-threshold rule to identify unusually high daily costs.
- Provides sample mode for demonstrations without real provider credentials or infrastructure changes.
- Uses a browser dashboard and documented REST API for access to project functions.
- Requires explicit approval for stop/delete operations and records action outcomes for traceability.
- Uses PostgreSQL persistence and Alembic migrations to manage stored information and schema evolution.
- Supports local development using Docker Compose and an optional Terraform sandbox.

Disadvantages and limitations

- Forecasts depend on the quantity and quality of historical cost data; short or irregular histories can produce weak estimates.
- The linear-trend fallback cannot represent all seasonal patterns, abrupt changes, or provider billing adjustments.
- The threshold-based anomaly detector may flag legitimate spending changes or miss anomalies that do not cross the threshold.
- Live data collection requires correct credentials, permissions, provider configuration, and billing-export setup where applicable.
- Provider APIs may have delays, quotas, schema changes, and different billing semantics.
- Stop and delete operations can interrupt workloads or permanently remove data; user review and least-privilege access remain essential.
- The application does not itself guarantee savings, calculate carbon emissions, or replace provider-native governance tools.
- The current project is a prototype and requires further load testing, security review, and validation with representative production data before organizational deployment.

Applications

- Academic demonstration of API development, relational database design, time-series analysis, and cloud SDK integration.
- Review of daily and service-level cloud expenditure by developers or small engineering teams.
- Identification of unusual cost observations for further investigation.
- Preparation of short-term cost estimates to support planning discussions.
- Inventory review for stopped or idle resources that may need human attention.
- Testing of controlled infrastructure actions in a safe simulated environment.
- Prototype for future extensions such as budget alerts, role-based approval, and more detailed reporting.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Cloud Cost Prediction and Optimization brings together cost-data ingestion, normalized persistence, forecasting, anomaly detection, dashboard presentation, and controlled resource actions in a single software project. The implementation is organized into phases so that the API and database foundation can be developed before data collection and analytics are added. Sample mode supports repeatable demonstrations without requiring live cloud access.

The forecasting workflow uses Prophet when its requirements are met and a linear-trend fallback otherwise. The anomaly detector uses a transparent threshold derived from prior observations. These approaches make the results accessible for review, but they should be evaluated against representative billing histories before being treated as operational decision criteria. The approval-based remediation design and audit logging provide safeguards for resource-changing actions, while still requiring careful human review.

The project provides a practical learning experience in backend development, data modelling, analytics, cloud integrations, deployment, and software quality. Its principal contribution is an integrated prototype and a clear workflow for turning cost records into reviewable information and controlled actions. No measured cost-saving or forecast-accuracy claim is made in this report because validated numerical results were not supplied in the project materials.

7.2 Future Scope

- Evaluate forecasting models using rolling-origin validation and suitable error metrics such as MAE and RMSE.
- Add configurable budgets, alert notifications, and scheduled analysis runs.
- Improve anomaly detection with robust statistics and methods that account for service-specific patterns.
- Add role-based access control, approval history, and separate permissions for viewing and changing resources.
- Provide dry-run previews that show the exact proposed action and resource metadata before approval.
- Improve audit export and report generation for reviews and academic demonstrations.

- Add data-quality checks for currency differences, delayed billing records, missing values, and revised invoices.
- Introduce automated tests for provider adapters using mocked API responses and expand integration testing.
- Add resource utilization metrics where permissions and provider APIs make them available, so idle classification can use stronger evidence than state alone.
- Conduct a security review and operational evaluation before using the prototype with important workloads.

CHAPTER 8: TIME LINE CHART OF PROJECT

The following timeline reflects the phase structure documented in the supplied project files. The durations are indicative planning periods for the academic project and should be updated to match the team's actual approved schedule.

Milestone	Main activities	Indicative period	Deliverable / checkpoint
1. Requirement analysis and design	Define problem, goals, scope, database entities, API boundaries, and safety requirements.	Weeks 1–2	Approved requirements and initial architecture.
2. Platform foundation	Create FastAPI app, health route, database models, migrations, adapter contract, Docker configuration, and contract tests.	Weeks 3–4	Foundation and local startup workflow.
3. Sandbox infrastructure design	Prepare Terraform modules, variables, outputs, tags, and safe defaults; validate configuration.	Weeks 5–6	Optional infrastructure configuration and validation notes.
4. Cost and resource ingestion	Implement provider adapters, sample adapter, normalization, persistence, and ingestion routes.	Weeks 7–9	Stored cost records and resource candidates.
5. Forecasting and anomaly analysis	Prepare daily series, implement Prophet/fallback forecasting, thresholds, persistence, and analysis routes.	Weeks 10–11	Forecast and anomaly endpoints.
6. Dashboard and remediation	Build dashboard views, resource listing, explicit approval checks, action execution, and audit logging.	Weeks 12–13	Dashboard and controlled-action workflow.
7. Testing and report preparation	Exercise sample mode, review edge cases, verify document structure, update diagrams and bibliography.	Weeks 14–15	Final review, report, and presentation materials.

Figure 8.1: Project timeline can be represented as a Gantt chart using the milestone sequence above. Exact calendar dates should be entered once the project schedule is confirmed.

BIBLIOGRAPHY

[1] Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts. https://otexts.com/fpp3/

[2] Taylor, S. J., & Letham, B. (2018). Forecasting at Scale. The American Statistician, 72(1), 37–45. https://doi.org/10.1080/00031305.2017.1380080

[3] Box, G. E. P., J GitHub Actions , G. M., Reinsel, G. C., & Ljung, G. M. (2015). Time Series Analysis: Forecasting and Control (5th ed.). Wiley.

[4] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. ACM Computing Surveys, 41(3), Article 15. https://doi.org/10.1145/1541880.1541882

[5] Hodge, V. J., & Austin, J. (2004). A Survey of Outlier Detection Methodologies. Artificial Intelligence Review, 22, 85–126. https://doi.org/10.1023/B:AIRE.0000045502.10941.a9

[6] Ahmad, S., Lavin, A., Purdy, S., & Agha, Z. (2017). Unsupervised Real-Time Anomaly Detection for Streaming Data. Neurocomputing, 262, 134–147. https://doi.org/10.1016/j.neucom.2017.04.070

[7] Nawrocki, P., & Sus, W. (2022). Anomaly detection in the context of long-term cloud resource usage planning. Knowledge and Information Systems, 64, 2689–2711. https://doi.org/10.1007/s10115-022-01721-5

[8] Smendowski, M., & Nawrocki, P. (2024). Optimizing multi-time series forecasting for enhanced cloud resource utilization based on machine learning. Knowledge-Based Systems, 304, 112489. https://doi.org/10.1016/j.knosys.2024.112489

[9] Intrusion detection in cloud computing based on time series anomalies utilizing machine learning. (2023). Journal of Cloud Computing. https://doi.org/10.1186/s13677-023-00491-x

[10] Meta. Prophet documentation. https://facebook.github.io/prophet/

[11] Amazon Web Services. Using the AWS Cost Explorer API. https://docs.aws.amazon.com/cost-management/latest/userguide/ce-api.html

[12] Microsoft. Azure Cost Management documentation. https://learn.microsoft.com/azure/cost-management-billing/

[13] Google Cloud. Set up Cloud Billing data export to BigQuery. https://cloud.google.com/billing/docs/how-to/export-data-bigquery-setup

[14] Amazon Web Services. AWS Trusted Advisor documentation. https://docs.aws.amazon.com/awssupport/latest/user/trusted-advisor.html

[15] Microsoft. Azure Advisor documentation. https://learn.microsoft.com/azure/advisor/

[16] Google Cloud. Recommender documentation. https://cloud.google.com/recommender/docs

[17] FinOps Foundation. FinOps Framework. https://www.finops.org/framework/

[18] FastAPI. Official documentation. https://fastapi.tiangolo.com/

[19] SQLAlchemy. SQLAlchemy 2.0 documentation. https://docs.sqlalchemy.org/

[20] PostgreSQL Global Development Group. PostgreSQL documentation. https://www.postgresql.org/docs/

[21] Alembic. Official documentation. https://alembic.sqlalchemy.org/

[22] pandas development team. pandas documentation. https://pandas.pydata.org/docs/

[23] Docker. Docker Compose documentation. https://docs.docker.com/compose/

[24] HashiCorp. Terraform documentation. https://developer.hashicorp.com/terraform/docs

[25] pytest. Official documentation. https://docs.pytest.org/

APPENDIX

Appendix A: Sample API workflow

The following workflow summarizes a typical local demonstration. The commands are illustrative and should be executed only after the project environment has been configured.

1. Start PostgreSQL and the FastAPI application using the documented local or Docker Compose setup.
2. Open the health endpoint and confirm that the API responds.
3. Call the ingestion endpoint with sample mode enabled to create predictable cost and resource records.
4. Run the analysis endpoint to generate forecasts and anomaly records from the stored sample history.
5. Open the dashboard and review costs, forecasts, anomalies, and resources.
6. For remediation demonstration, choose a sample resource and submit an explicitly approved action. Confirm that the operation is simulated and inspect the audit log.

Appendix B: Important safety notes

- Never commit cloud secrets, access keys, passwords, or local environment files to source control.
- Use least-privilege identities and verify the target account and resource before any live action.
- Do not run infrastructure provisioning commands without reviewing the plan and understanding possible charges.
- Use sample mode for demonstrations and testing when live cloud access is not necessary.
- Deletion can be irreversible. Verify backups, ownership, tags, and approval before requesting deletion.
- Treat forecasts and anomalies as decision-support outputs that require human interpretation.